

ASHISH TIRKEY

+91 7808788191 | ashish.tirkey2399@gmail.com | Hyderabad, India | [linkedin.com/in/ashish-tirkey-9a69661b](https://www.linkedin.com/in/ashish-tirkey-9a69661b)

Backend Software Engineer | Distributed Systems & Microservices | AI/LLM
Infrastructure | Observability

PROFESSIONAL SUMMARY

Backend Software Engineer with 4 years at Kore.ai building distributed microservices and production infrastructure for an enterprise conversational AI platform. Owned systems spanning configuration distribution across **100+ services**, production observability, and request-path LLM middleware, using **Node.js**, **Java**, **MongoDB**, **Redis**, and **AWS**.

TECHNOLOGIES

Java, JavaScript, C++, Node.js, Express.js, Spring Boot, MongoDB, PostgreSQL, Redis, RabbitMQ, AWS, Docker, GitHub Actions, Loki, Grafana, PagerDuty, OpenAI API

WORK EXPERIENCE

Software Engineer | Kore.ai | Backend Services Team

Jan 2024 – May 2026

- Owned three backend systems end to end, each from initial design through deployment and post-launch iteration: a configuration management platform, a production alerting service, and an LLM guardrail middleware layer.
- Designed and owned a distributed configuration platform used by **8 internal engineering teams**, propagating versioned configuration across **100+ services** and removing a DevOps bottleneck that slowed every team's release cycle. Config changes that previously took hours of manual coordination became self-service, and config-related deployment incidents dropped **~30%** over the first 6 months. Profiling config-read endpoints and applying **MongoDB** compound indexing with **Redis** LRU/TTL caching cut response times **~35%** without infrastructure scaling.
- Built an LLM guardrail layer in the synchronous request path after policy-violating AI responses and PII in outputs were reaching users undetected. Evaluated across two user groups totalling **10,000+ users**, it reduced policy-violating outputs **~35%** with no meaningful drop in session completion rate, giving the product team confidence to expand the DialogGPT integration.
- The alerting service changed how the team handled production incidents. Detection previously relied on manual log checks with MTTD averaging over 30 minutes. After rollout, **MTTD dropped to 18 minutes** and P1 resolution time fell from around 2 hours to **under 45 minutes**, meaningfully reducing the on-call burden.
- Collaborated with DevOps, QA, the ML/NLP team, and Product on requirements for the config and guardrail systems. Conducted **50+ code reviews**, wrote architecture decision records, and automated the deployment pipeline via **GitHub Actions** on **AWS EC2**, cutting deployment time from ~20 minutes to **under 5**.

Associate Software Engineer | Kore.ai | Backend Services Team

Jul 2022 – Jan 2024

- Contributed to API development, data modelling, and background job processing for the core conversational AI platform using **Node.js**, **Express.js**, and **MongoDB**.
- Wrote the initial version of the configuration service that later became the full orchestration platform, giving early visibility into the config management problem before taking ownership of the redesign.

Software Engineering Intern | Samsung Research Institute

May 2022 – Jul 2022

- Built backend REST APIs in **Spring Boot** and a **Java** regression test suite automating **40+ test scenarios**, cutting manual QA time by roughly **60%** for the team.

Configuration Management and Orchestration Platform

Stack: Node.js, Express.js, MongoDB, Redis, Docker, AWS EC2, GitHub Actions

- The core design challenge was propagating config changes to **100+ services** without restarts. Used a REST API with JSON schema validation to reject malformed configs before storage, **append-only versioned documents in MongoDB** so rollback was a re-point rather than a state mutation, and **Redis Pub/Sub** for in-memory propagation without restarts.
- The key tradeoff was **Redis Pub/Sub vs Kafka**. Redis was chosen for operational simplicity, but Pub/Sub is fire-and-forget: busy services under load silently drop messages. Addressed this with a **reconciliation** mechanism where each service periodically compares its config version against the source of truth and self-heals on mismatch, giving **eventual consistency** without Kafka's overhead.

Alert Sender: Observability and Intelligent Alerting System

Stack: Node.js, Loki, LogQL, Grafana, Slack API, PagerDuty, RabbitMQ, Redis

- Ran scheduled **LogQL** queries against **Loki** on a configurable interval per error pattern. Deduplication initially failed because raw error strings included dynamic values like request IDs, making every occurrence look unique; fixed by normalising strings with regex before hashing to extract **stable fingerprints**.
- Chose **scheduled polling over streaming** deliberately, since 60-second intervals were sufficient at the team's scale and kept the system simple enough for one engineer to own and debug in production.

LLM Guardrail Middleware and AI Safety Pipeline

Stack: Node.js, Redis, OpenAI API, External Guardrail APIs

- The middleware sat in the synchronous request path, making **latency** the central constraint. Profiling showed the external guardrail API call dominated and that many requests carried near-identical prompts; fixed by parallelising independent checks and caching results on a **normalised prompt hash in Redis**, bringing added latency from ~400ms to **under 100ms**.
- The policy engine was built as a **pluggable registry** so new LLM providers integrate without touching core logic, with violations routed to a **human review queue** rather than dropped.

EDUCATION

B.Tech, Electronics and Communication Engineering | NIT Jamshedpur

2018 – 2022

LEARNING AND DEVELOPMENT

- Personal projects: building with the **OpenAI Assistants API** and **LlamaIndex** to develop practical RAG pipeline experience, complementing the AI safety work done at Kore.ai.